Reinforcement Learning versus Classic Stochastic Control Solutions for a Simple Liquidation Problem

Lewis Wilcher

October 2026

Abstract

We show the robustness of a closed-form optimal control derived against changing market conditions to those setup in the initial derivation of our control. We also compare these results to a competing alternative of reinforcement learning which achieves an 'optimal' solution to our liquidation problem through a learning technique despite no assumptions of the environment of the market.

1 Introduction

Within quantitative finance there are applications of stochastic control within algorithmic trading. By setting what would be our value function, a measurement of success within our scenario and task, we are able to derive optimal solutions with respect to certain controls that we are able to change during an environment with stochastic processes. These problems hold similar qualities like that a quadratic term of our control with our HJB equation derived, this allows us to differentiate with respect to our control and find the the control which maximizes our utility or value function. The solutions derived in this scenario are analytical and therefore have strengths of lack of need for any more computationally heavy work and can be deployed easily through time. But they are not without drawbacks.

As with many classic stochastic modeling problems, we make assumptions around the dynamics of the processes within our environment. Specifically in order to find an analytical solution to our problem with ease we may 'manufacture' a quadratic term in our control variable(s), but this clearly may not reflect the environment as correctly. This may lead to us solving an optimal set of controls that doesn't solve the issue or environment we had earlier set out to solve. Leaving this point, within the deployment of such an optimal control there are also the requirements of accurate estimations of the parameters that determine our control, mis-specifying our parameters can lead to our utility or final value being non-optimal. It is clear to see that within the noise of financial markets, the true estimation of market parameters is likely to hold a certain amount of error.

A way of bypassing all of this trouble would be to implement an entirely different method which requires no over described assumptions of our environment, reinforcement learning. The deployment of an RL agent has the strength of requiring no analytical solution but at the cost of a what could be a computationally heavy training period. RL agents work within a framework of states z_j and actions a_j, creating a distribution of conditional probabilities for actions given being at certain states.

2 Parameter Misspecifcation with Stochastic Control Liquidation Problems

We begin by setting up a classic liquidation problem which might be equivalent to a portfolio manager needing to liquidate a position for a particular stock S_t from its current inventory Q_0 to a final position Q_T.

Our model follows the following dynamics:

$$dQ_t = \nu_t \, dt, \tag{1}$$
$$dS_t = b \, \nu_t \, dt + \sigma \, dB_t, \tag{2}$$
$$dX_t = -\left(S_t + k \, \nu_t\right) \nu_t \, dt, \tag{3}$$

with initial conditions $Q_0 > 0$, S_0 and $X_0 = 0$. The processes and parameters are:

Q_t the inventory, i.e. the number of shares held at time t. We start with Q_0 shares and wish to sell them down by the horizon T.

ν_t the trading rate, in shares per unit time. This is our control. A negative ν_t means we are selling, so equation (21) reduces the inventory.

S_t the market price of the stock. It is driven by B_t, a standard Brownian motion representing random market moves, whose size is set by the volatility $\sigma > 0$.

b the permanent price impact. Trading at rate ν_t moves the price by $b \, \nu_t \, dt$, and this move stays in the price for all later trades. With $b > 0$, selling pushes the price down.

X_t the cash account, starting from $X_0 = 0$. We receive cash when we sell ($\nu_t < 0$).

k the temporary price impact, $k > 0$. The price we actually trade at is $S_t + k \, \nu_t$, which is worse than the quoted price when we sell. Unlike b, this effect is not carried into S_t and applies only to the trade being made. It makes the cost of trading, $k \, \nu_t^2 \, dt$, quadratic in the trading rate, which penalises liquidating too quickly.

At the horizon T any shares we have not sold are valued at a penalised price $S_T - \alpha Q_T$, where $\alpha > 0$ is the penalty on leftover inventory. Our terminal wealth is therefore

$$W_T = X_T + Q_T \left(S_T - \alpha Q_T\right). \tag{4}$$

We measure success by exponential (constant absolute risk aversion) utility, with risk aversion $\gamma > 0$, so that the aim is to choose the trading rate ν to maximise

$$\sup_{\nu} \; \mathbb{E}\left[-\exp\left(-\gamma \, W_T\right)\right], \tag{5}$$

where the supremum is taken over admissible controls. A larger γ means we care more about reducing the spread of outcomes and not only the average.

2.1 Value function and HJB equation

The value function is the best expected utility we can achieve from time t onward, given the current cash, price and inventory:

$$V(t,x,s,q) = \sup_{\nu} \; \mathbb{E}\Big[-\exp\left(-\gamma\, W_T\right) \;\Big|\; X_t = x,\; S_t = s,\; Q_t = q \Big]. \tag{6}$$

By the dynamic programming principle, V satisfies the Hamilton–Jacobi–Bellman (HJB) equation

$$V_t + \sup_{\nu} \left\{ \nu\, V_q + b\, \nu\, V_s - (s + k\,\nu)\, \nu\, V_x \right\} + \tfrac{1}{2}\sigma^2 V_{ss} = 0, \tag{7}$$

with terminal condition

$$V(T,x,s,q) = -\exp\left(-\gamma\left(x + q\left(s - \alpha q\right)\right)\right). \tag{8}$$

The terms in the braces are the drifts of Q, S and X multiplied by the corresponding derivatives of V. The only second-order term is $\frac{1}{2}\sigma^2 V_{ss}$, because S is the only process driven by Brownian motion.

The expression inside the supremum is quadratic in ν:

$$-k\, V_x\, \nu^2 + \left(V_q + b\, V_s - s\, V_x\right) \nu.$$

Since $V_x > 0$ for this utility, the coefficient of ν^2 is negative, so the expression is concave in ν and the supremum is attained by taking a derivative with respect to ν and rearranging to find our ν^*:

$$\nu^* = \frac{V_q + b\, V_s - s\, V_x}{2\, k\, V_x}. \tag{9}$$

Substituting ν^* back into (7) removes the supremum:

$$\begin{cases} V_t + \dfrac{\left(V_q + b\, V_s - s\, V_x\right)^2}{4\, k\, V_x} + \tfrac{1}{2}\sigma^2 V_{ss} = 0, \\ V(T,x,s,q) = -\exp\left(-\gamma\left(x + q\left(s - \alpha q\right)\right)\right). \end{cases} \tag{10}$$

2.2 Optimal control

Because the utility is exponential, we look for a solution of the form

$$V(t,x,s,q) = -\exp\left(-\gamma\left(x + qs + \varphi(t,q)\right)\right), \qquad \varphi(T,q) = -\alpha q^2, \tag{11}$$

where the terminal condition on φ reproduces the terminal condition on V. Substituting into the first-order condition, the price s cancels and the optimal trading rate becomes

$$\nu^* = \frac{\varphi_q + b\,q}{2k}. \tag{12}$$

Substituting the ansatz into the HJB equation leaves a PDE for φ in (t,q) only. Since the terminal condition and every term in this PDE are quadratic in q, we can take $\varphi(t,q) = -A(t)\,q^2$, which reduces the PDE to the Riccati equation

$$A'(t) = \frac{\left(2A(t) - b\right)^2}{4k} - \tfrac{1}{2}\gamma\sigma^2, \qquad A(T) = \alpha. \tag{13}$$

This is separable and can be solved in closed form. Writing $\psi(t) = 2A(t) - b$ and $\lambda = \sigma\sqrt{2k\gamma}$, the solution is

$$\psi(t) = \lambda\,\frac{1 + R(t)}{1 - R(t)}, \qquad R(t) = \frac{2\alpha - b - \lambda}{2\alpha - b + \lambda}\,e^{-\lambda(T-t)/k}, \tag{14}$$

which is well defined on $[0,T]$ whenever $2\alpha - b > -\lambda$ (in particular whenever $2\alpha \geq b$). Since $\varphi_q = -2A(t)q$, the optimal control is

$$\boxed{\nu_t^* = -\frac{\psi(t)}{2k}\,Q_t = -\frac{\lambda}{2k}\,\frac{1 + R(t)}{1 - R(t)}\,Q_t} \tag{15}$$

The optimal strategy therefore sells a time-dependent fraction of the current inventory per unit time, independent of the price and the cash. It depends on the permanent impact b through $2\alpha - b$ and on the volatility σ through λ, and these are the two parameters we mis-specify below.

2.3 Monte Carlo set-up

To study the effect of mis-specified parameters, we compute the optimal control of the previous section using believed values $\hat{b}$ and $\hat{\sigma}$ in place of b and σ, and apply it in a market driven by the true values $b = 0.01$ and $\sigma = 1$. The market is simulated with an Euler scheme on $n = 500$ time steps of size $\Delta t = T/n$ with $T = 1$:

$$\begin{aligned} \nu_j &= -\frac{\hat{\psi}(t_j)}{2k}\,Q_j, \\ Q_{j+1} &= Q_j + \nu_j\,\Delta t, \\ S_{j+1} &= S_j + b\,\nu_j\,\Delta t + \sigma\,\Delta B_j, \\ X_{j+1} &= X_j - \left(S_j + k\,\nu_j\right)\nu_j\,\Delta t, \end{aligned} \tag{16}$$

where $\hat{\psi}$ is the function ψ computed with $\hat{b}$ and $\hat{\sigma}$, and $\Delta B_j \sim \mathcal{N}(0, \Delta t)$ are independent. We use $N = 20{,}000$ paths and the same Brownian increments for every strategy, so that differences between strategies come from the strategy and not from the sampling noise. The remaining parameters are $Q_0 = 10$, $S_0 = 100$, $X_0 = 0$, $k = 0.1$, $\alpha = 0.5$ and $\gamma = 0.25$.

We misspecify one parameter at a time, with the other (and k) known correctly: the believed value is $\hat{b} = r_b\, b$ or $\hat{\sigma} = r_\sigma\, \sigma$ with the ratio r taken over 31 equally spaced values in $[0, 3]$, where $r = 1$ is the correctly specified strategy. For each value we compute the terminal wealth $W_T = X_T + Q_T(S_T - \alpha Q_T)$ on every path and record its sample mean and standard deviation. We also estimate the expected utility achieved by the strategy,

$$\mathbb{E}\left[-e^{-\gamma W_T}\right] \approx \frac{1}{N} \sum_{i=1}^{N} \left(-e^{-\gamma W_T^{(i)}}\right), \tag{17}$$

which we refer to as the value function.

2.4 Conclusion

We can see that the changes in our permanent price impact b have had very little impact upon the optimality of our strategy however there is clear effects within the misspecified sigma.

There is also a clear illustration of how the risk aversion within our value function works. It is helpful to see that when r_σ is at 1 our value function is maximised however the expectation of W_T is not and nor is the standard deviation, showing that our value function shows that we are maximising our wealth while being risk averse to too large a spread on our final return.

Finally the bottom-right graph within figure 1 clearly shows that our optimal strategy falls off quickly when our believed $\hat{\sigma}$ grows a multiple larger than the true market σ.

This is all clear proof of where using an RL agent would avoid the need to parameterise a model in this sense.

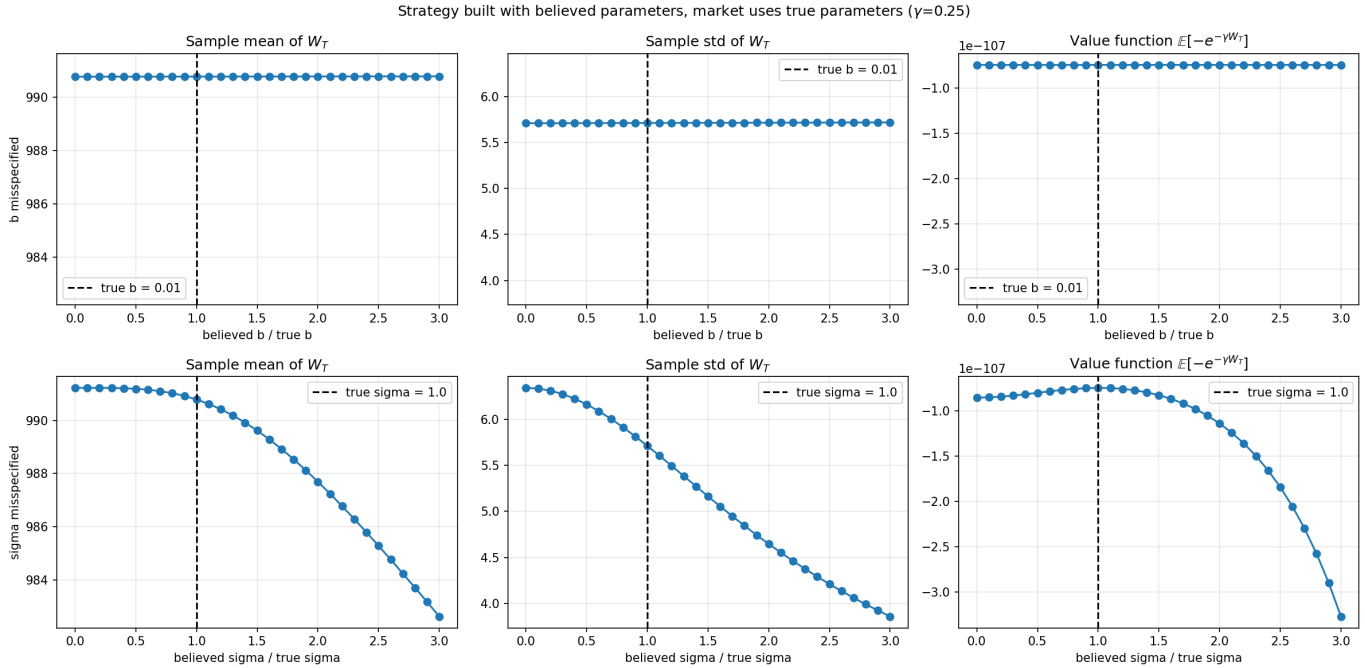


Figure 1: Sample mean and standard deviation of W_T and the value function $\mathbb{E}\left[-e^{-\gamma W_T}\right]$ (third column, vertical axis multiplied by 10^{-107}) for a strategy built with a believed permanent impact $\hat{b}$ (top row) or a believed volatility $\hat{\sigma}$ (bottom row), applied in a market driven by the true parameters. The dashed line marks the correctly specified case, and $\gamma = 0.25$. Each column has the same vertical axis in both rows.

3 Reinforcement learning

The previous sections solved the liquidation problem by dynamic programming: we wrote the HJB equation, guessed the structure of the value function and solved an ODE. Reinforcement learning (RL) attacks the same optimisation problem, choose a feedback strategy to maximise expected terminal utility, but finds the strategy by experience instead of by solving an equation. An agent repeatedly acts in a simulated market, observes the outcome, and adjusts the numbers that define its strategy so that decisions which worked out better than expected become more likely. The algorithm never uses the coefficients of the dynamics; it only needs to be able to draw the next state of the market given the current state and an action. This section sets up the general framework, derives the policy gradient on which the method is based, and then describes the agent we implemented to solve the liquidation problem. The agent is a policy gradient method written from scratch (it uses no reinforcement-learning library), so that every step of the derivation appears explicitly in the code.

3.1 Notation

$j,\ n$	Index of a time step, $j = 0, \dots, n$, and the number of steps in an episode.
$s_j,\ \mathcal{S}$	State at step j, and the set of states. The state includes the time remaining, so no separate time index is needed.
$a_j,\ \mathcal{U}$	Action taken at step j, and the set of allowed actions.
r_j	Reward received after the action at step j.
$P(s' \mid s, a)$	Transition kernel: the law of the next state given the state and the action (the market dynamics).
$\pi(a \mid s)$	Policy: the probability (density) of taking action a in state s.
θ	Parameter vector (the "weights") of the policy; π_θ is the policy they define.
τ	Trajectory $(s_0, a_0, s_1, \dots, a_{n-1}, s_n)$ of one episode; $p_\theta(\tau)$ is its probability under π_θ.
G_j	Return from step j: the sum of the rewards from step j to the end of the episode, $G_j = \sum_{i=j}^{n-1} r_i$; $G = G_0$.
$J(\theta)$	Objective: the expected return of the policy π_θ.
$V^\pi(s)$	State value: expected return starting in s and following π. V^* is the optimal value.
$\mathcal{Q}^\pi(s, a)$	Action value: expected return starting in s, taking a first and following π afterwards.
$\mathcal{A}^\pi(s, a)$	Advantage, $\mathcal{Q}^\pi(s, a) - V^\pi(s)$.
$\mu_\theta(s)$	Mean of the Gaussian policy used for exploration (Section 3.6).
$V_\zeta(s)$	Critic: a second function, with weights ζ, that approximates $V^\pi(s)$.
$\hat{\mathcal{A}}_j$	Estimate of the advantage at step j, computed from data.
$\delta,\ \delta_\zeta$	Learning rates: the sizes of the gradient steps on the policy weights θ and the critic weights ζ.
N	Number of episodes played in one iteration; episode i has states $s_j^{(i)}$, actions $a_j^{(i)}$ and advantage estimates $\hat{\mathcal{A}}_j^{(i)}$.
$\hat{g}$	Estimate of the gradient $\nabla_\theta J$ computed from a batch.
ς	Standard deviation of the exploration noise (fixed).
ϕ_j	Vector of features of the state at step j (Section 3.9); the policy mean is $\mu_\theta(s_j) = \theta^\top \phi_j$.
$c_j,\ c_{\max}$	Trading speed chosen at step j ($\nu_j = -c_j Q_j$) and its upper bound.
θ_κ	Long-run level of the impact process κ (the parameter called θ in the market model).
w_{ref}	Reference wealth used to scale the reward.
$K,\ m$	Number of learning iterations, and the index of the current iteration.

3.2 The set-up: agent, environment and policy

An episode is one complete liquidation of the position over $[0, T]$, discretised into n steps. At each step j the following happens:

1. The agent observes the state s_j.
2. It chooses an action a_j according to its *policy* $\pi(a \mid s_j)$.
3. The environment returns a reward r_j and moves to a new state $s_{j+1} \sim P(\cdot \mid s_j, a_j)$.

This is a finite-horizon *Markov decision process*: the law of (r_j, s_{j+1}) depends on the past only through (s_j, a_j). The correspondence with the control problem is:

Stochastic control	Reinforcement learning
State $(t, X_t, S_t, Q_t, \kappa_t)$	State s_j
Control ν_t	Action a_j
Feedback strategy $\nu_t = \nu(t, \text{state})$	Policy $\pi(a \mid s)$
Controlled SDE	Transition kernel $P(s_{j+1} \mid s_j, a_j)$
Terminal utility $-e^{-\gamma W_T}$	Reward r_j and return G
Value function V	Value function V^π
HJB equation	Bellman equation
Solve a PDE/ODE	Learn from sampled trajectories

A *policy* is a rule that maps a state to a probability distribution over actions. We choose it to be randomised, for a reason explained in Section 3.6, and we make it depend on a vector of numbers θ, written π_θ. For example θ can be the weights of a neural network or, as in our implementation, the coefficients of a linear function of a few features of the state. One run of the market under π_θ produces a trajectory $\tau = (s_0, a_0, s_1, a_1, \ldots, a_{n-1}, s_n)$ with probability

$$p_\theta(\tau) = p(s_0) \prod_{j=0}^{n-1} \pi_\theta(a_j \mid s_j)\, P(s_{j+1} \mid s_j, a_j). \tag{18}$$

Two sources of randomness appear in (18): the agent's own random choice of action, and the market's response. The agent is *model-free*: it never sees the formula for P, only samples from it.

3.3 The objective function

The quantity the agent tries to maximise is the expected total reward of an episode,

$$J(\theta) = \mathbb{E}_{\tau \sim p_\theta}\big[G(\tau)\big] = \mathbb{E}_{\tau \sim p_\theta}\left[\sum_{j=0}^{n-1} r_j\right] = \int p_\theta(\tau)\, G(\tau)\, \mathrm{d}\tau, \tag{19}$$

and the goal is $\max_\theta J(\theta)$. In our application the reward is zero at every step except the last, where it is a utility of terminal wealth, so $J(\theta)$ is the expected utility $\mathbb{E}[-e^{-\gamma W_T}]$ (up to a positive constant) obtained by following π_θ. Maximising it over all policies is exactly the control problem of the previous sections; maximising it over the family $\{\pi_\theta\}$ is the same problem restricted to strategies the agent can represent.

3.4 State value and action value

To decide which decisions are good, we need to measure the future reward that follows them. Define the *state value* and the *action value* of a policy π:

$$V^\pi(s) = \mathbb{E}_\pi\left[G_j \mid s_j = s\right], \tag{20}$$

$$\mathcal{Q}^\pi(s,a) = \mathbb{E}_\pi\left[G_j \mid s_j = s,\ a_j = a\right]. \tag{21}$$

$V^\pi(s)$ is the expected remaining reward when starting in s and following π from then on. $\mathcal{Q}^\pi(s,a)$ is the same, except that the first action is forced to be a. They are related by averaging over the first action, $V^\pi(s) = \mathbb{E}_{a\sim\pi(\cdot|s)}[\mathcal{Q}^\pi(s,a)]$, and $V^\pi(s_n) = 0$ at the end of the episode. The value of the starting state is the objective: $J = V^\pi(s_0)$.

Bellman expectation equations. The return satisfies $G_j = r_j + G_{j+1}$. Conditioning first on the action drawn from π and then on the next state drawn from P, and using that by the Markov property the future after $s_{j+1} = s'$ does not depend on how we arrived at s', we obtain

$$V^\pi(s) = \mathbb{E}_{a\sim\pi(\cdot|s),\ s'\sim P(\cdot|s,a)}\Big[r + V^\pi(s')\Big], \tag{22}$$

$$\mathcal{Q}^\pi(s,a) = \mathbb{E}_{s'\sim P(\cdot|s,a)}\Big[r + \mathbb{E}_{a'\sim\pi(\cdot|s')}\mathcal{Q}^\pi(s',a')\Big], \tag{23}$$

where r is the reward of the step from s to s'. In words: the value of a state is the reward received now plus the value of the state we land in, averaged over the agent's choice of action and the market's response.

3.5 Bellman optimality equations

Let $V^*(s) = \sup_\pi V^\pi(s)$ and $\mathcal{Q}^*(s,a) = \sup_\pi \mathcal{Q}^\pi(s,a)$ be the best achievable values. By the dynamic programming principle, an optimal policy must act optimally from whatever state it reaches next, which gives

$$V^*(s) = \sup_a\ \mathbb{E}_{s'\sim P(\cdot|s,a)}\Big[r + V^*(s')\Big], \tag{24}$$

$$\mathcal{Q}^*(s,a) = \mathbb{E}_{s'\sim P(\cdot|s,a)}\Big[r + \sup_{a'}\mathcal{Q}^*(s',a')\Big], \tag{25}$$

with $V^*(s) = \sup_a \mathcal{Q}^*(s,a)$ and an optimal policy $\pi^*(s) = \arg\max_a \mathcal{Q}^*(s,a)$.

The optimality equations define the answer, but they are hard to use directly here. Solving them requires P, which the agent does not know, and a maximisation over the action at every state, which is awkward when the action is a continuous trading speed. *Policy-gradient* methods avoid both: they parameterise the policy as π_θ and improve θ directly by gradient ascent on $J(\theta)$. The rest of this section explains how.

3.6 Exploration and the learning of the weights

Suppose the agent always took the single action it currently believes is best. Then every episode would reveal the outcome of that one action only, and there would be nothing to compare it with. To learn that another action is better, the agent must *try* it. We therefore make the policy random, so that it tries different actions in the same state. The simplest choice for a continuous action is a Gaussian,

$$\pi_\theta(a \mid s) = \mathcal{N}\big(a;\ \mu_\theta(s),\ \varsigma^2\big), \tag{26}$$

where $\mu_\theta(s)$ is a function of the state with weights θ (a linear function of features of the state in our case, Section 3.9): it is the action the agent currently believes is best, and ς controls how widely it explores around it. A large ς gathers broader information but wastes reward on poor actions, and a small ς explores little. (In our implementation ς is a fixed constant.) This is the *exploration–exploitation trade-off*. When the trained agent is deployed, it uses the mean action $\mu_\theta(s)$ with no added noise.

Learning then proceeds as follows. The agent plays episodes with the current weights θ. Each action a_j taken in a state s_j is followed by an outcome. If the outcome was *better than what is typical from* s_j, we change θ so that $\pi_\theta(a_j \mid s_j)$ increases, which for the Gaussian policy moves the mean $\mu_\theta(s_j)$ towards a_j. If it was worse, we change θ so that $\pi_\theta(a_j \mid s_j)$ decreases. Two ingredients are needed to turn this into a formula: a number that scores each action as better or worse than typical (the advantage), and the direction in θ-space that raises the probability of a given action (the gradient of $\log \pi_\theta$).

3.7 The advantage: scoring actions

The raw return G_j is a poor score for the action a_j. A large G_j may just mean the agent was in a favourable state, and every action would have done well. The advantage removes this by comparing with the average from the same state:

$$\mathcal{A}^\pi(s,a) = \mathcal{Q}^\pi(s,a) - V^\pi(s). \tag{27}$$

It is the extra return obtained by choosing a instead of an action drawn from π, and by construction $\mathbb{E}_{a\sim\pi(\cdot|s)}[\mathcal{A}^\pi(s,a)] = 0$. An action with $\mathcal{A}^\pi > 0$ is better than the policy's average action in that state and should become more likely; an action with $\mathcal{A}^\pi < 0$ should become less likely, and the size of $\mathcal{A}^\pi$ says by how much.

Estimating the advantage. Neither $\mathcal{Q}^\pi$ nor V^π is known. From one sampled episode we observe the return G_j, and $\mathbb{E}[G_j \mid s_j, a_j] = \mathcal{Q}^\pi(s_j, a_j)$, so G_j is an unbiased (but noisy) sample of the action value. For the state value we train a second function, the *critic* $V_\zeta(s)$ with weights ζ, to predict the return by least squares:

$$\min_{\zeta} \ \mathbb{E}\Big[\big(V_\zeta(s_j) - G_j\big)^2\Big]. \tag{28}$$

The advantage estimate is then

$$\hat{\mathcal{A}}_j = G_j - V_\zeta(s_j). \tag{29}$$

It is positive when the episode did better than the critic expected from s_j and negative when it did worse.

3.8 Derivation of the policy gradient

We now derive the direction in which to change θ to increase $J(\theta)$. Since θ only affects the reward through $p_\theta(\tau)$ in (19), differentiating J means differentiating a probability distribution.

Step 1: log-derivative trick. Assuming we may differentiate under the integral,

$$\begin{aligned}\nabla_\theta J(\theta) &= \int G(\tau)\, \nabla_\theta p_\theta(\tau)\, \mathrm{d}\tau = \int G(\tau)\, p_\theta(\tau)\, \nabla_\theta \log p_\theta(\tau)\, \mathrm{d}\tau \\ &= \mathbb{E}_{\tau\sim p_\theta}\Big[G(\tau)\, \nabla_\theta \log p_\theta(\tau)\Big],\end{aligned} \tag{30}$$

where we used $\nabla_\theta p = p\, \nabla_\theta \log p$. The gradient is now an *expectation under* p_θ, and so can be estimated by simulation.

Step 2: the dynamics drop out. By (18),

$$\log p_\theta(\tau) = \log p(s_0) + \sum_j \log \pi_\theta(a_j \mid s_j) + \sum_j \log P(s_{j+1} \mid s_j, a_j).$$

Only the middle sum depends on θ, so

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau\sim p_\theta}\left[\sum_{j=0}^{n-1} \nabla_\theta \log \pi_\theta(a_j \mid s_j)\; G(\tau)\right]. \tag{31}$$

The coefficients of the market dynamics appear nowhere in this formula. The vector $\nabla_\theta \log \pi_\theta(a \mid s)$, the *score*, points in the direction in θ-space that makes the action a more likely in state s. Equation (31) says: increase the probability of the actions taken in an episode, in proportion to the episode's reward.

Step 3: reward-to-go. For $i < j$, the reward r_i is determined by the history up to s_j, which is fixed before a_j is drawn, and $\mathbb{E}[\nabla_\theta \log \pi_\theta(a_j \mid s_j) \mid \text{history}] = 0$ by the argument of Step 4. These terms therefore vanish, and $G(\tau)$ can be replaced by the reward-to-go G_j:

$$\nabla_\theta J(\theta) = \mathbb{E}\left[\sum_{j=0}^{n-1} \nabla_\theta \log \pi_\theta(a_j \mid s_j)\, G_j\right]. \tag{32}$$

Step 4: baseline. For any function $b(s)$ that does not depend on the action,

$$\begin{aligned}\mathbb{E}_{a\sim\pi_\theta(\cdot|s)}\big[\nabla_\theta \log \pi_\theta(a \mid s)\, b(s)\big] &= b(s) \int \nabla_\theta \pi_\theta(a \mid s)\, \mathrm{d}a \\ &= b(s)\, \nabla_\theta \int \pi_\theta(a \mid s)\, \mathrm{d}a = b(s)\, \nabla_\theta 1 = 0. \end{aligned} \tag{33}$$

Hence G_j can be replaced by $G_j - b(s_j)$ without changing the expectation. A good b makes the term smaller and less noisy. Taking $b = V^{\pi_\theta}$, and using $\mathbb{E}[G_j - V^{\pi_\theta}(s_j) \mid s_j, a_j] = \mathcal{Q}^{\pi_\theta}(s_j, a_j) - V^{\pi_\theta}(s_j) = \mathcal{A}^{\pi_\theta}(s_j, a_j)$, gives the *policy gradient theorem*

$$\boxed{\nabla_\theta J(\theta) = \mathbb{E}_{\tau\sim p_\theta}\left[\sum_{j=0}^{n-1} \nabla_\theta \log \pi_\theta(a_j \mid s_j)\, \mathcal{A}^{\pi_\theta}(s_j, a_j)\right]} \tag{34}$$

The score of each action taken is multiplied by its advantage: actions that did better than average are reinforced, and actions that did worse are discouraged.

Step 5: estimate from samples. Run the current policy to generate N episodes, with step j of episode i having state $s_j^{(i)}$, action $a_j^{(i)}$ and advantage estimate $\hat{\mathcal{A}}_j^{(i)}$ from (29). The sample average

$$\hat{g} = \frac{1}{N} \sum_{i=1}^{N} \sum_{j=0}^{n-1} \nabla_\theta \log \pi_\theta\big(a_j^{(i)} \mid s_j^{(i)}\big)\, \hat{\mathcal{A}}_j^{(i)} \tag{35}$$

is an estimate of (34). It remains unbiased whatever the critic is, because the critic only acts as a baseline (Step 4); a more accurate critic reduces its variance. It only requires the gradient of $\log \pi_\theta$ at the actions actually taken. For the Gaussian policy with fixed ς the score is

$$\nabla_\theta \log \pi_\theta(a \mid s) = \frac{a - \mu_\theta(s)}{\varsigma^2}\, \nabla_\theta \mu_\theta(s), \tag{36}$$

so an action above the current mean ($a > \mu_\theta$) with positive advantage moves $\mu_\theta(s)$ up towards it, and with negative advantage moves it down, away from it.

Step 6: gradient ascent. The gradient (34) is an expectation over trajectories of an environment we cannot write down, and J is a nonlinear function of θ, so we cannot solve $\nabla_\theta J = 0$ for θ. Instead, we use the first-order approximation $J(\theta+\Delta) \approx J(\theta) + \nabla_\theta J(\theta)^\top \Delta$: the step $\Delta = \delta \nabla_\theta J(\theta)$ with small $\delta > 0$ increases J by approximately $\delta \|\nabla_\theta J\|^2 \geq 0$. With the estimate $\hat{g}$ this is the *update of the weights*

$$\boxed{\theta_{m+1} = \theta_m + \delta\, \hat{g}_m} \tag{37}$$

It is gradient *ascent* (a plus sign) because J is maximised. Each component of θ is changed by δ times the corresponding component of $\hat{g}_m$. The learning proceeds by repeating:

1. Play N episodes with the current policy π_{θ_m}, storing states, actions and rewards.
2. Compute the returns G_j and the advantage estimates $\hat{\mathcal{A}}_j = G_j - V_\zeta(s_j)$; fit the critic V_ζ to the returns.
3. Compute $\hat{g}_m$ from (35).
4. Update $\theta_{m+1} = \theta_m + \delta \hat{g}_m$.
5. Discard the data and return to step 1 with the new policy.

The data must be regenerated at each iteration because (34) is an expectation under the *current* policy. The iteration reaches a point where $\nabla_\theta J \approx 0$, which is a local maximum of J with no guarantee of a global one.

3.9 The liquidation problem as a reinforcement learning problem

We now specify the environment, the policy, the reward and the learning algorithm used for the liquidation problem. The same agent is trained in two markets, the one with constant temporary impact (in which the closed-form control is optimal) and the one with stochastic temporary impact κ.

Environment. An episode is one liquidation of Q_0 shares over $[0, T]$, discretised into $n = 100$ steps of length $\Delta t = T/n$. The state of the market evolves, for a trading rate ν_j, according to the Euler scheme

$$\begin{aligned} X_{j+1} &= X_j - (S_j + \kappa_j \nu_j)\, \nu_j\, \Delta t, \\ S_{j+1} &= S_j + b\, \nu_j\, \Delta t + \sigma\, \Delta B_j, \\ Q_{j+1} &= Q_j + \nu_j\, \Delta t, \\ \kappa_{j+1} &= \kappa_j + \beta(\theta_\kappa - \kappa_j)\, \Delta t + \eta\, \Delta Z_j, \end{aligned} \tag{38}$$

with $\Delta B_j, \Delta B_j^\perp \sim \mathcal{N}(0, \Delta t)$ independent and $\Delta Z_j = \rho\, \Delta B_j + \sqrt{1-\rho^2}\, \Delta B_j^\perp$. In the market with constant impact the last equation is replaced by $\kappa_{j+1} = \kappa_j$,

and $\kappa_j \equiv k$ is the constant impact coefficient. The process κ is not bounded. The initial state is $(X_0, S_0, Q_0, \kappa_0) = (0, 100, 10, 0.1)$ and the terminal wealth is $W_T = X_n + Q_n(S_n - \alpha Q_n)$, with α the terminal inventory penalty of the earlier sections.

Action. As in the closed-form solution $\nu_t^* = -\frac{\psi(t)}{2k} Q_t$, the agent chooses the *fraction of the remaining inventory sold per unit time*, the trading speed c_j:

$$\nu_j = -\operatorname{clip}(c_j,\, 0,\, c_{\max})\, Q_j, \qquad c_{\max} = 10, \tag{39}$$

where $\operatorname{clip}(x, l, r)$ restricts x to the interval $[l, r]$. The agent therefore only ever sells, at a rate proportional to its inventory. This restricts the agent to strategies of the same form as the closed-form control, with c allowed to depend on time and on κ. In the market with constant impact this form contains the optimal strategy. In the market with stochastic impact we do not claim that the optimal strategy has this form: the agent maximises the objective within this class of strategies.

State and features. The inventory enters only through the factor Q_j in ν_j, so the speed c_j is allowed to depend on the time and on the current impact level. The agent observes these through the feature vector

$$\phi_j = \big(1,\; x_j,\; x_j^2,\; x_j^3,\; \tilde{\kappa}_j,\; \tilde{\kappa}_j\, x_j\big)^\top \in \mathbb{R}^6, \qquad x_j = \frac{2j}{n} - 1, \quad \tilde{\kappa}_j = \frac{\kappa_j}{\theta_\kappa} - 1. \tag{40}$$

The variable x_j runs from -1 at the start to $+1$ at the end of the episode, and the powers of x_j let the speed vary smoothly in time. The closed-form speed $\psi(t)/(2k)$ is a smooth function of time, which a cubic polynomial in x approximates with a maximum error of about 0.3 for the values of γ used here, while the speed itself ranges from about 1 to 5. The feature $\tilde{\kappa}_j$ is the impact level relative to its long-run value and $\tilde{\kappa}_j x_j$ lets its effect change through time; in the market with constant impact $\tilde{\kappa}_j = 0$ and these two features vanish.

Policy. The policy is Gaussian in the speed, with a mean that is linear in the features:

$$c_j \sim \pi_\theta(\,\cdot \mid s_j) = \mathcal{N}\big(\mu_\theta(s_j),\; \varsigma^2\big), \qquad \mu_\theta(s_j) = \theta^\top \phi_j, \qquad \varsigma = 0.5. \tag{41}$$

The six weights θ are initialised at $\theta = (3, 0, 0, 0, 0, 0)$, so that before learning the agent sells at the constant speed 3 in every state. The score of Section 3.8 is then

$$\nabla_\theta \log \pi_\theta(c_j \mid s_j) = \frac{c_j - \theta^\top \phi_j}{\varsigma^2}\, \phi_j, \tag{42}$$

evaluated at the sampled speed c_j before clipping. After training, the agent is deployed with no noise, $c_j = \operatorname{clip}(\theta^\top \phi_j, 0, c_{\max})$.

Reward. The reward is zero at every step except the last, where it is the utility of terminal wealth,

$$r_{n-1} = -\exp\Big(\min\big\{-\gamma\,(W_T - w_{\text{ref}}),\ 20\big\}\Big), \tag{43}$$

so that every return G_j equals the single terminal reward $G = r_{n-1}$. Without w_{ref} and the cap this is the utility $-e^{-\gamma W_T}$. The reference wealth w_{ref} is the sample mean of W_T under the closed-form strategy in the same market and for the same γ; it multiplies the utility by the positive constant $e^{\gamma w_{\text{ref}}}$, which does not change which strategy is best but keeps the numbers readable. The cap at e^{20} limits the effect of very rare, very bad episodes. Hence $J(\theta) = e^{\gamma w_{\text{ref}}}\,\mathbb{E}\big[-e^{-\gamma W_T}\big]$ up to the cap.

Critic and advantage. The critic is linear in the same features, $V_\zeta(s_j) = \zeta^\top \phi_j$, with the weights ζ initialised at zero. In each iteration the advantage of step j of episode i is

$$\hat{\mathcal{A}}_j^{(i)} = G^{(i)} - \zeta^\top \phi_j^{(i)}, \tag{44}$$

and the critic is moved towards the observed returns by a gradient step on the squared error,

$$\zeta \leftarrow \zeta + \delta_\zeta\,\frac{1}{Nn}\sum_{i=1}^{N}\sum_{j=0}^{n-1}\hat{\mathcal{A}}_j^{(i)}\,\phi_j^{(i)}, \qquad \delta_\zeta = 0.05. \tag{45}$$

The critic sees only the time and κ, and not the inventory or the wealth accumulated so far. It is therefore a crude baseline, but by Step 4 of Section 3.8 this affects only the variance of the gradient estimate and not its mean.

Gradient estimate and update. The advantages of the whole iteration (all Nn steps) are standardised to mean 0 and standard deviation 1,

$$\tilde{\mathcal{A}}_j^{(i)} = \frac{\hat{\mathcal{A}}_j^{(i)} - \overline{\mathcal{A}}}{\text{sd}(\hat{\mathcal{A}}) + 10^{-8}}, \tag{46}$$

where $\overline{\mathcal{A}}$ and $\text{sd}(\hat{\mathcal{A}})$ are their sample mean and standard deviation. Subtracting a constant is a change of baseline (Step 4) and dividing by a positive number only rescales the gradient, so the direction of the update is unchanged, while its size no longer depends on the scale of the rewards (which is very different for $\gamma = 0.1$ and $\gamma = 0.5$). The gradient estimate is the average over all steps of all episodes,

$$\hat{g} = \frac{1}{Nn}\sum_{i=1}^{N}\sum_{j=0}^{n-1}\frac{c_j^{(i)} - \theta^\top \phi_j^{(i)}}{\varsigma^2}\,\phi_j^{(i)}\,\tilde{\mathcal{A}}_j^{(i)}, \tag{47}$$

which is (35) with the sum over steps replaced by an average (a constant factor n, absorbed into the learning rate). The weights are updated by gradient ascent,

$$\theta_{m+1} = \theta_m + \delta_m\,\hat{g}_m, \qquad \delta_m = \delta_0\left(1 - 0.9\,\frac{m}{K}\right), \quad \delta_0 = 1. \tag{48}$$

The learning rate decreases linearly from δ_0 to $0.1\,\delta_0$ over the K iterations, so that the steps are large at first and small near the end, when the noise in $\hat{g}_m$ would otherwise make θ wander around the optimum.

Training loop. One iteration $m = 0, \ldots, K-1$ consists of:

1. Simulate $N = 256$ episodes in parallel with fresh, independent noise, using the current policy with exploration noise. Record the features ϕ_j and the sampled speeds c_j.
2. Compute the terminal wealth W_T of each episode and its reward (43).
3. Compute the advantages, update the critic, standardise the advantages, and compute $\hat{g}_m$ from (47).
4. Update $\theta_{m+1} = \theta_m + \delta_m \hat{g}_m$ and discard the data.

We use $K = 1500$ iterations, i.e. 384,000 training episodes, for each market and each value of γ.

Evaluation. After training, the agent and the closed-form control are applied, without exploration noise, to the *same* 10,000 test paths of $(\Delta B_j, \Delta Z_j)$, which are generated once and are independent of the training noise. For each strategy we compute the sample mean and standard deviation of W_T and the expected utility

$$\hat{U} = \frac{1}{10{,}000} \sum_{p=1}^{10{,}000} -\exp\left(-\gamma \left(W_T^{(p)} - w_{\mathrm{ref}}\right)\right), \tag{49}$$

which is closer to zero for a better strategy. Using the same paths for both strategies removes the sampling noise from their comparison.

Settings.

Steps per episode n, horizon T	100, 1
Risk aversions γ	0.1, 0.25, 0.5
Episodes per iteration N	256
Iterations K	1500
Learning rate δ_m	1, decreasing linearly to 0.1
Critic learning rate δ_ζ	0.05
Exploration standard deviation ς	0.5 (in units of trading speed)
Initial policy	constant speed 3
Speed bound $c_{\max}$, reward cap exponent	10, 20
Number of weights in θ and in ζ	6 each
Test paths	10,000, common to the control and the agent

4 Results

This section presents the output of the experiment described in Section 3.9, in two price models: the constant-volatility (Black–Scholes) price used so far, and a Heston price with stochastic volatility.

4.1 Experimental set-up

Markets. In each price model two markets are considered, M1 (constant impact $\kappa \equiv k$) and M2 (stochastic impact κ_t following the OU process), and each is run for the risk aversions $\gamma \in \{0.1, 0.25, 0.5\}$.

Heston price. In the Heston model the price volatility is $\sqrt{v_t}$, where the variance follows a square-root process correlated with the price noise:

$$\begin{aligned} dS_t &= b\,\nu_t\,dt + \sqrt{v_t}\,dB_t, \\ dv_t &= a_v\,(\bar{v} - v_t)\,dt + \xi\sqrt{v_t}\,dB^v_t, \qquad d\langle B, B^v\rangle_t = \rho_{sv}\,dt, \end{aligned} \tag{50}$$

with $v_0 = \bar{v} = \sigma^2 = 1$, $a_v = 3$, $\xi = 1$ and $\rho_{sv} = -0.5$. The mean of v_t is then the constant variance σ^2 of the Black–Scholes price, and the Feller condition $2a_v\bar{v} \geq \xi^2$ holds ($6 \geq 1$). Everything else in the market (wealth, inventory and, in M2, the process κ_t) is as in Section 3.9. The Euler scheme (38) is used with the equation for S replaced by

$$\begin{aligned} S_{j+1} &= S_j + b\,\nu_j\,\Delta t + \sqrt{v_j^+}\,\Delta B_j, \\ v_{j+1} &= v_j + a_v\,(\bar{v} - v_j^+)\,\Delta t + \xi\sqrt{v_j^+}\,\Delta B^v_j, \qquad v_j^+ = \max(v_j, 0), \end{aligned} \tag{51}$$

where $\Delta B^v_j = \rho_{sv}\,\Delta B_j + \sqrt{1-\rho_{sv}^2}\,\Delta B^{(2)}_j$ and $\Delta B^{(2)}_j \sim \mathcal{N}(0, \Delta t)$ is independent of ΔB_j and $\Delta B^{\perp}_j$. The initial state is $(X_0, S_0, Q_0, \kappa_0, v_0) = (0, 100, 10, 0.1, 1)$.

Strategies. In every market we compare two strategies:

- the *closed-form control* $\nu^*_t = -\frac{\psi(t)}{2k}Q_t$ of the constant-impact, constant-volatility model, computed for the given γ with $\sigma = \sqrt{\bar{v}} = 1$. It is optimal only in M1 with the Black–Scholes price. In M2 it is applied unchanged, which also acts as a test of model misspecification, and in the Heston price it does not use the variance v_t, so it is not optimal in either market;
- the *policy gradient agent* of Section 3.9, with the same six features (so it does not observe v_t), the same policy, reward and settings. It is trained separately in each market and for each γ, on episodes simulated from that market, and then deployed without exploration noise.

Evaluation. After training, both strategies are applied to the same 10,000 test paths of the driving noise, which are independent of the training data. For each market and each γ we report the sample mean and the sample standard deviation of terminal wealth W_T, and the expected utility

$$\hat{U} = \frac{1}{10{,}000} \sum_{p=1}^{10{,}000} -\exp\Big(-\gamma\big(W_T^{(p)} - w_{\mathrm{ref}}\big)\Big), \tag{52}$$

where w_{ref} is the sample mean of W_T under the closed-form control in the same market and for the same γ (Section 3.9). Higher values of $\hat{U}$ are better, and the maximum possible value is 0.

The training curves show the same three quantities (mean and standard deviation of W_T, and expected utility) over the course of training. Each column is one value of γ. The blue curve is computed from the $N = 256$ training episodes of each iteration, which are played *with* exploration noise, averaged over blocks of 25 consecutive iterations; the utility is the average of the reward (43), including the cap on the exponent. The dashed horizontal line is the value of the closed-form control on the test paths.

4.2 Black–Scholes price

Figure 2 shows the test-path results and Figures 3 and 4 the training curves.

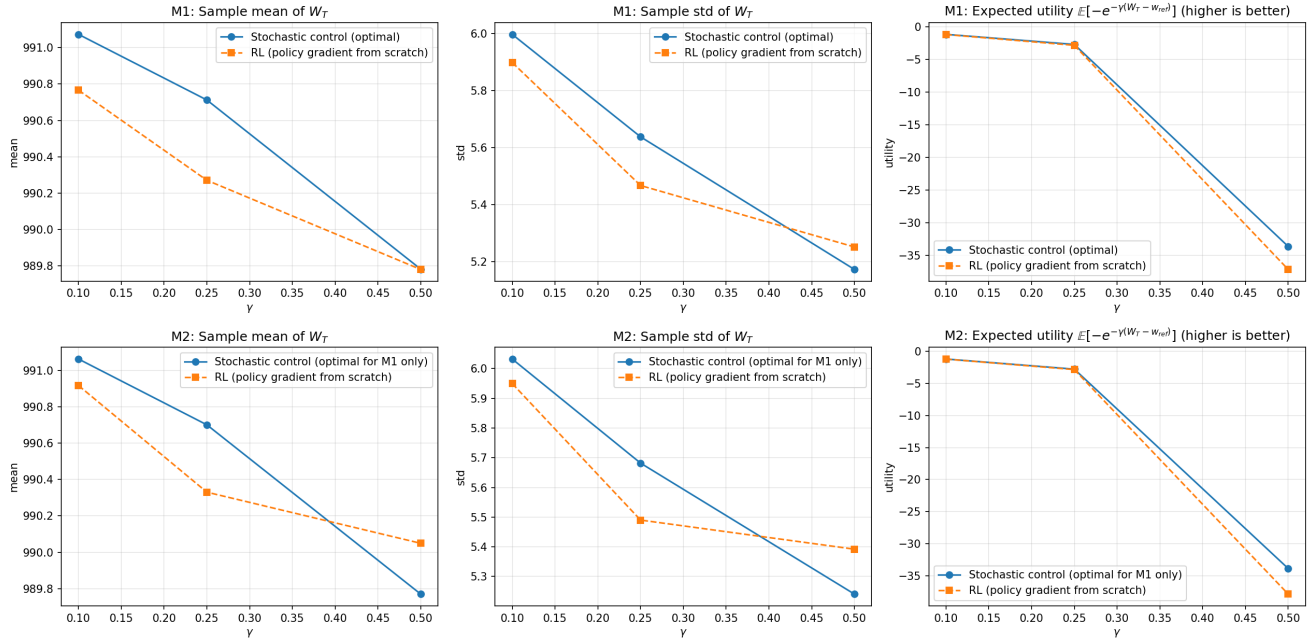


Figure 2: Black–Scholes price: closed-form control and policy gradient agent on the 10,000 common test paths, for $\gamma \in \{0.1, 0.25, 0.5\}$. Top row: M1 (constant impact). Bottom row: M2 (stochastic impact). Columns: sample mean of W_T, sample standard deviation of W_T, and expected utility $\hat{U}$.

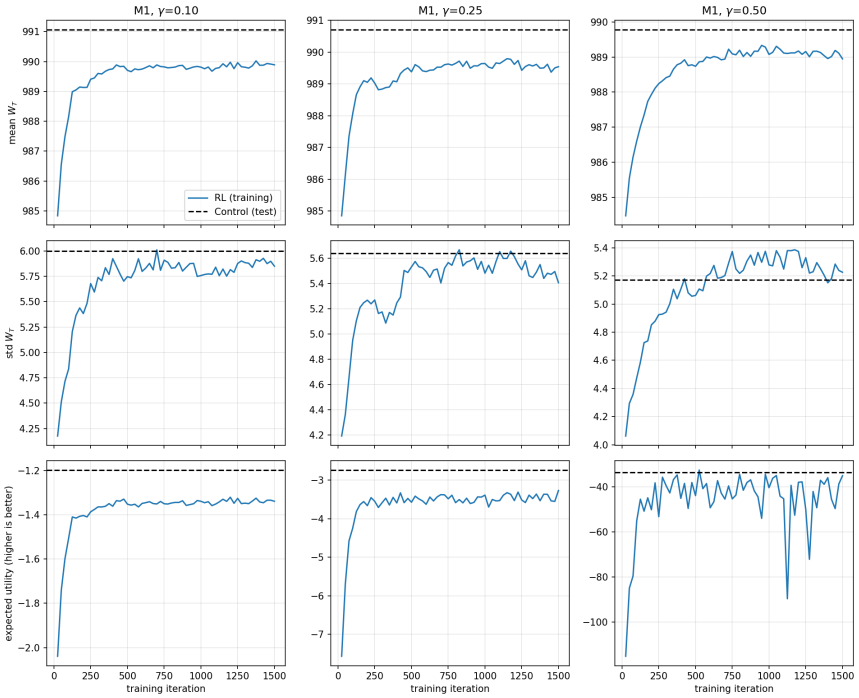


Figure 3: Black–Scholes price, M1: training curves of the policy gradient agent for $\gamma = 0.1, 0.25, 0.5$. Rows: mean of W_T, standard deviation of W_T and expected utility over the training episodes; the dashed line is the closed-form control on the test paths.

4.3 Heston price

Figure 5 shows the test-path results and Figures 6 and 7 the training curves.

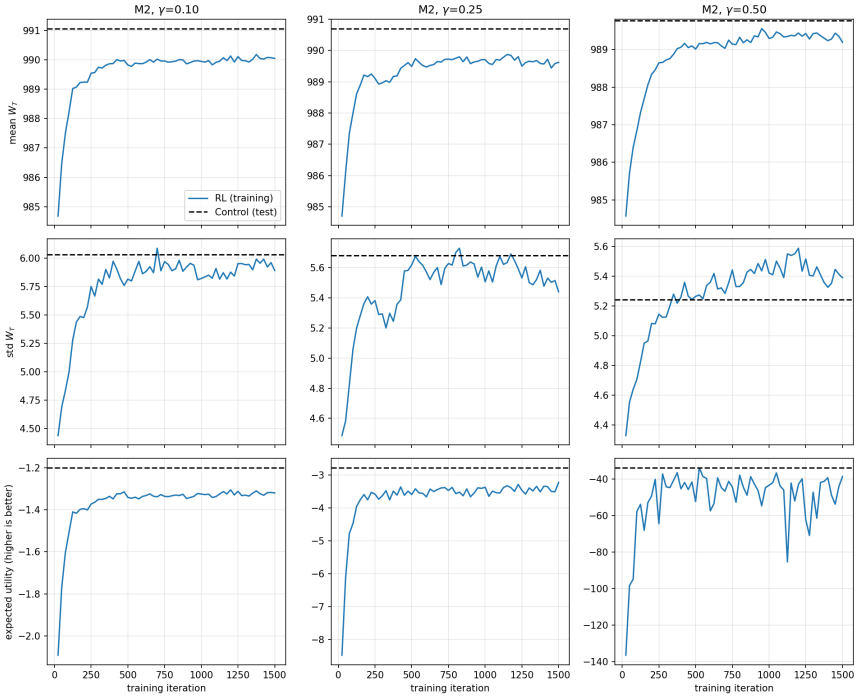


Figure 4: Black–Scholes price, M2: training curves of the policy gradient agent. Rows and lines as in Figure 3.

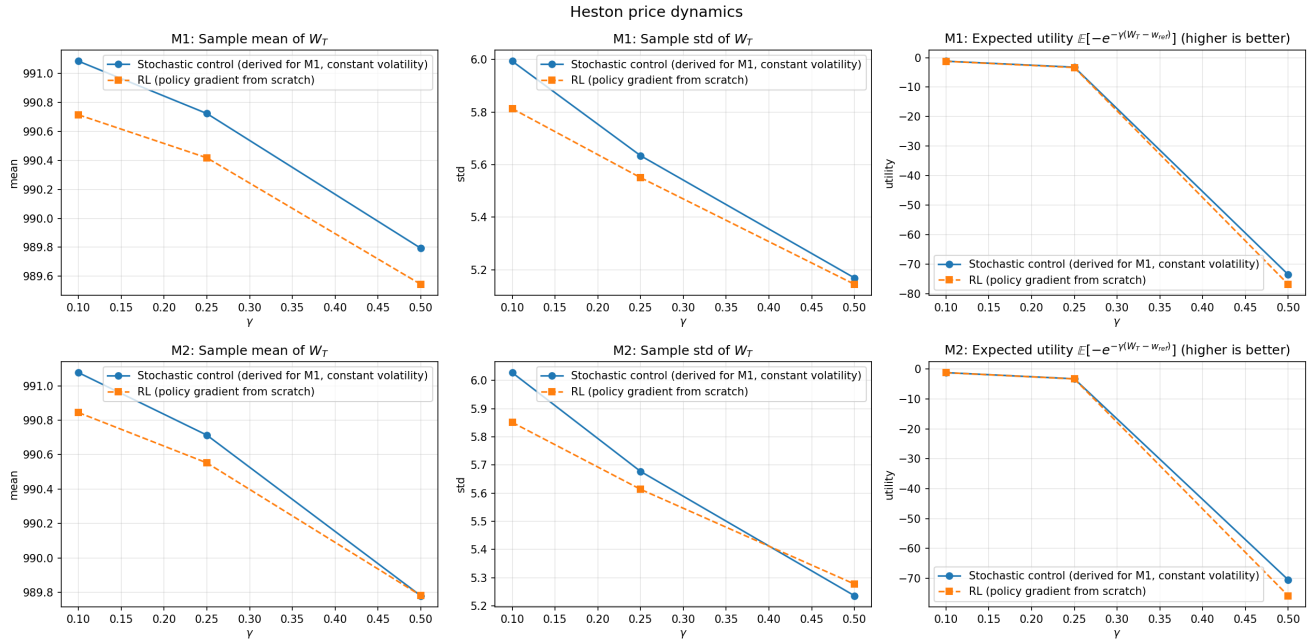


Figure 5: Heston price: closed-form control and policy gradient agent on the 10,000 common test paths. Layout as in Figure 2.

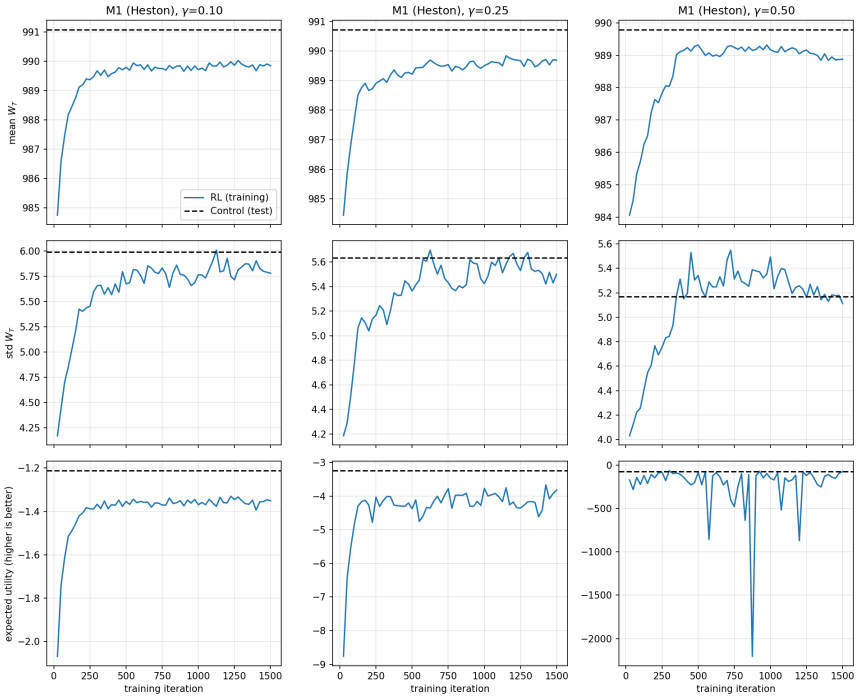


Figure 6: Heston price, M1: training curves of the policy gradient agent. Rows and lines as in Figure 3.

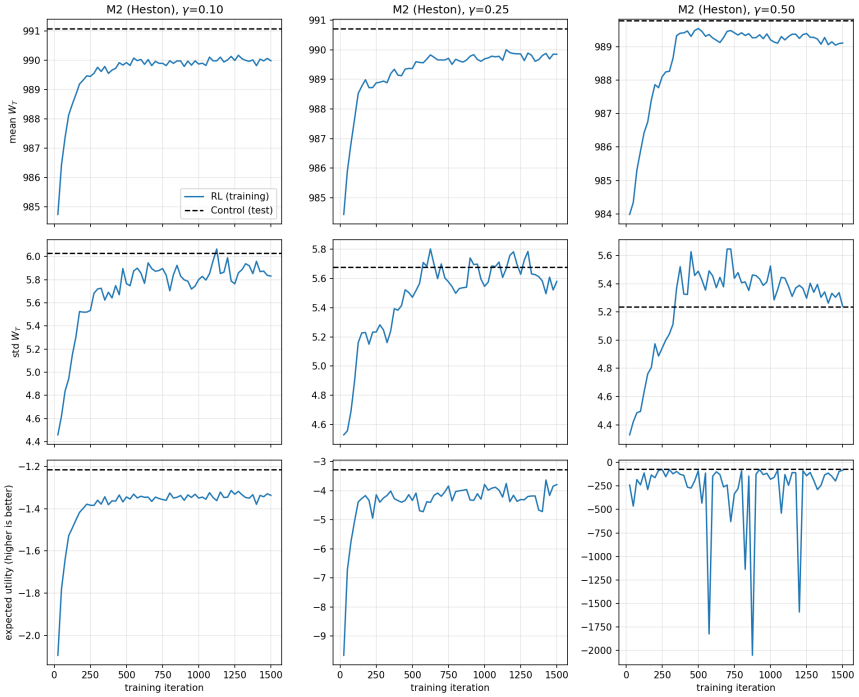


Figure 7: Heston price, M2: training curves of the policy gradient agent. Rows and lines as in Figure 3.

4.4 Numerical Results

			Closed-form control			RL agent		
Price model	Market	γ	mean W_T	std W_T	$\hat{U}$	mean W_T	std W_T	$\hat{U}$
Black–Scholes	M1	0.10	991.073	5.997	−1.197	990.766	5.899	−1.227
		0.25	990.712	5.638	−2.741	990.271	5.468	−2.875
		0.50	989.782	5.173	−33.665	989.781	5.252	−37.186
	M2	0.10	991.064	6.032	−1.200	990.918	5.950	−1.212
		0.25	990.701	5.682	−2.776	990.331	5.490	−2.847
		0.50	989.769	5.241	−33.865	990.049	5.392	−37.858
Heston	M1	0.10	991.086	5.994	−1.212	990.715	5.813	−1.243
		0.25	990.724	5.634	−3.243	990.418	5.551	−3.362
		0.50	989.793	5.169	−73.599	989.544	5.145	−76.881
	M2	0.10	991.077	6.028	−1.214	990.845	5.851	−1.228
		0.25	990.714	5.677	−3.268	990.552	5.614	−3.313
		0.50	989.780	5.236	−70.516	989.782	5.277	−75.830

Table 1: Closed-form control and policy gradient agent on the 10,000 common test paths: sample mean and standard deviation of W_T and expected utility $\hat{U} = \mathbb{E}\big[-e^{-\gamma(W_T - w_{\text{ref}})}\big]$, for each price model, market and risk aversion γ. These are the values plotted in Figures 2 and 5.

5 Concluding Thoughts

First I think its important to note Figure 2, that shows a clear comparison of where our derived optimal trading speed is compared to our RL trained agent. Our optimal policy clearly reaches a higher expected utility as made obvious in our training curves in Figure 3, but it is clear that the RL agent is 'near-optimal' in this sense.

We then turn to both our M2 model in Figure 2 and M1 model in Figure 5, both these situations show one clear model misspecification. For the M2 model mentioned, we introduced a stochastic price impact dynamics which has a mean-reverting tendancy. For each expected utility our optimal policy achieved a lower 'score' (Expected Utility) than on the M1 simulation (as we would expect) and interestingly our trained agent achieved a higher score than on M1 for each risk aversion other than $\gamma = 0.5$, as shown clearly in Table 1. This closed the gap between the two policies however our derived policy still produced greater results. Looking to now our M1 model with a Heston type price dynamics of our stock, we might expect that due to our closed-form control performance having clear correlation with the accuracy of choosing our volatility parameter, we may expect that now our RL agent would 'take the lead'. However this is not the case and once again despite there being very close results. It is clear that RL agent is unable to produce the single optimal policy but for a set of well-behaved dynamics it is able to produce what we have called 'near-optimal' or maybe optimal to a certain degree of error.

Finally we look at now two model misspecifications with a stochastic κ dynamics and also the heston price dynamics. However simply put the RL agent fails to outperform our closed-form control based on the criteria of our chosen value function. This clearly shows that our derived optimal control policy is robust to changes or inaccuracies in our modelling assumptions, which is contrary to possible earlier thoughts where a Reinforcement Learning Agent may be able to outperform due to the lack of need for a 'well set up' HJB equation. I would argue that this is clearly a spectrum of how far you trust your model dynamics in relation to the market. Despite us not finding the conditions where our RL agent may takeover from the closed-form control, it clearly would exist when the dynamics simulating the market are far from those set up to solve for our stochastic optimal control. I would also point out that an RL agent could be set up with a greater flexibility for a changing market regime. Even when deployed it would be possible to run a learning process in the background which would change the trained policy $\pi(a \mid s)$.